

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 12

Producer, Consumer and the Other Tools

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: Producer and consumer 2
2.1.1 Observe Answers (Part 1) 3
2.2 Part 2: The condition variable 4
2.2.1 Observe Answers (Part 2) 5
2.3 Part 3: Readers and writers 5
2.3.1 Observe Answers (Part 3) 6
2.4 Part 4: Build a deadlock, then remove it 7
2.4.1 Observe Answers (Part 4) 9

3 CONCLUSIONS **10**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

In Laboratory Work No. 11 one semaphore was used at a time: as a lock, as a signal and as a counter. The objective of this laboratory work is to make the tools work together: to solve the two classic synchronization problems (producer and consumer, readers and writers), to use a condition variable, and to build a deadlock and then remove it.

1.2 Theoretical Background

- **Producer and consumer.** A producer puts items into a buffer of limited size and a consumer takes them out. Three tools are needed: a semaphore that counts the free slots (the producer waits when the buffer is full), a semaphore that counts the filled slots (the consumer waits when the buffer is empty), and a mutex that protects the buffer itself. Every pipe in Unix works in this way, inside the kernel.

- **Condition variable.** A condition variable is a place where a thread sleeps until another thread says that something has changed. It has no memory: the state is kept in a normal variable protected by a mutex. The call `pthread_cond_wait` releases the mutex, sleeps, and takes the mutex again before it returns.

- **Readers and writers.** Many readers may read the shared data together, because reading does not change it, but a writer must be alone. If the readers never stop arriving, a writer may wait forever; this is called starvation.

- **Deadlock.** A deadlock appears when each thread of a group holds a resource and waits for a resource held by another thread of the group, so they form a circle and all of them wait forever. The circle cannot be formed if all the threads take the locks in the same order.

All the programs were compiled with `gcc -Wall -pthread`.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: Producer and consumer

In `prodcons.c` a fast producer puts 8 items into a buffer of 4 slots and a slow consumer takes them out. The semaphore `empty` counts the free slots (it starts at 4), the semaphore `full` counts the filled slots (it starts at 0), and the mutex `lock` protects the buffer.

```
#define SIZE  4
#define ITEMS 8

int buf[SIZE];
int in = 0, out = 0, count = 0;   /* count only for printing */
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);         /* wait for a free slot */
        pthread_mutex_lock(&lock);
        buf[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("put  %d   (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&full);          /* one more filled slot */
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);          /* wait for something to take */
        pthread_mutex_lock(&lock);
        int item = buf[out];
        out = (out + 1) % SIZE;
        count--;
        printf("            take %d   (in buffer: %d)\n", item, count);
        pthread_mutex_unlock(&lock);
        sem_post(&empty);         /* one more free slot */
        usleep(300000);           /* the consumer is slow */
    }
    return NULL;
}
```

In `main` the semaphores are initialized with `sem_init(&empty, 0, SIZE)` and `sem_init(&full, 0, 0)`, and the consumer thread is created before the producer. Figure 2.1 shows the output: the producer fills the buffer quickly, then it can put a new item only after the consumer takes one, so the number of items in the buffer never goes above 4. At the end the consumer empties the buffer.

```
put  1   (in buffer: 1)
put  2   (in buffer: 2)
            take 1   (in buffer: 1)
put  3   (in buffer: 2)
put  4   (in buffer: 3)
put  5   (in buffer: 4)
            take 2   (in buffer: 3)
put  6   (in buffer: 4)
            take 3   (in buffer: 3)
put  7   (in buffer: 4)
            take 4   (in buffer: 3)
put  8   (in buffer: 4)
            take 5   (in buffer: 3)
            take 6   (in buffer: 2)
            take 7   (in buffer: 1)
            take 8   (in buffer: 0)
```

Figure 2.1: Output of `prodcons`: the buffer never holds more than 4 items

For the third question, a copy of the program (`prodcons_swap.c`) was made, in which the first two lines of the producer loop are swapped: the mutex is locked before `sem_wait(&empty)`. It was run with `timeout 5`, which kills the program after 5 seconds and gives the exit code 124. Figure 2.2 shows that the program stops printing when the buffer is full and never finishes.

```
        pthread_mutex_lock(&lock);
        sem_wait(&empty);              /* wait for a free slot */
put  1   (in buffer: 1)
put  2   (in buffer: 2)
            take 1   (in buffer: 1)
put  3   (in buffer: 2)
put  4   (in buffer: 3)
put  5   (in buffer: 4)
exit code: 124
```

Figure 2.2: `prodcons_swap`: with the mutex locked before `sem_wait(&empty)` the program hangs (exit code 124)

2.1.1 Observe Answers (Part 1)

1. **Which call stops the producer?** The call `sem_wait(&empty)` at the beginning of the producer loop. The semaphore `empty` starts at 4 and counts the free slots: every `put` takes one unit, so after four items its value is 0 and the producer blocks there until the consumer takes an item and calls `sem_post(&empty)`.

2. **Who waits at the start, and in which call?** The consumer. The buffer is empty, so the semaphore `full` is 0 and the consumer blocks in `sem_wait(&full)`

until the producer puts the first item and calls `sem_post(&full)`. The producer does not wait, because `empty` starts at 4.

3. **What happens with the two lines swapped, and why?** The program hangs as soon as the buffer is full (Figure 2.2). The producer takes the mutex first and then blocks in `sem_wait(&empty)`, because there is no free slot, and it sleeps while still holding the mutex. The consumer is the only one that can free a slot, but it needs the same mutex to take an item, so it blocks in `pthread_mutex_lock`. Each thread waits for the other one: it is a deadlock. The rule is to wait on the semaphore first and to lock the mutex after.

2.2 Part 2: The condition variable

The program `condvar.c` solves the problem of `order.c` from Laboratory Work No. 11 (B must read only after A has written) in another way. The state is a normal variable, `ready`, protected by a mutex. The condition variable `changed` is only the place where B sleeps until A says that something has changed.

```c
char message[32];
int ready = 0;                    /* the condition itself */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t changed = PTHREAD_COND_INITIALIZER;

void *writer(void *arg) {
    sleep(1);
    pthread_mutex_lock(&lock);
    strcpy(message, "hello");
    ready = 1;
    printf("A: message written\n");
    pthread_cond_signal(&changed);  /* wake one waiter */
    pthread_mutex_unlock(&lock);
    return NULL;
}

void *reader(void *arg) {
    pthread_mutex_lock(&lock);
    /* while, not if: re-check after waking */
    while (!ready) {
        printf("B: not ready, I sleep\n");
        /* unlocks, sleeps, locks again */
        pthread_cond_wait(&changed, &lock);
    }
    printf("B: I read \"%s\"\n", message);
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

Figure 2.3 shows the output: B finds `ready` equal to 0 and sleeps, A writes the message one second later and signals, and only then B reads `"hello"`.

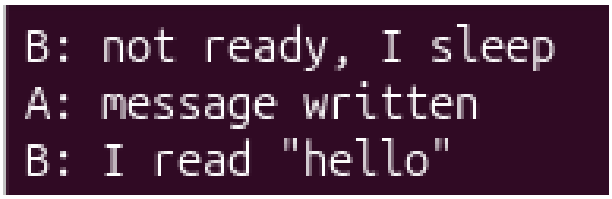


Figure 2.3: Output of `condvar`: B sleeps until A signals that the message is ready

For the second question, a copy of the program (`condvar_if.c`) was made, in which `while (!ready)` is replaced by `if (!ready)`. Its output is shown in Figure 2.4.



Figure 2.4: Output of `condvar_if` (`if` instead of `while`): the result is the same here

2.2.1 Observe Answers (Part 2)

1. **Which variable remembers that the message is ready?** The normal variable `ready`, protected by the mutex `lock`. The condition variable `changed` has no memory: it is only the place where B sleeps.

2. **Does it still work with `if`?** Yes, here it still works and prints the same three lines (Figure 2.4), because there is only one waiting thread and it is woken up exactly once, after `ready` was set to 1.

3. **Then why do we write `while`?** Because waking up does not guarantee that the condition is true. One reason is that `pthread_cond_wait` is allowed to return without any signal (a spurious wakeup), so the thread wakes up and `ready` is still 0. Another reason is that, with several waiting threads, another thread can run first and change the state back before this one gets the mutex again. The `while` checks the condition again and goes back to sleep if needed.

2.3 Part 3: Readers and writers

In `readers.c` there are three readers and two writers. Two semaphores are used: `room` (1 means that the room is free for a writer) and `rlock` (a lock around the counter `readers`). The first reader in locks the room, and the last one out unlocks it.

```
1  int value = 0;              /* the shared data */
2  int readers = 0;            /* how many are reading now */
3  sem_t rlock;                /* protects the counter readers */
4  sem_t room;                 /* 1 = the room is free for a writer */
5
6  void *reader(void *arg) {
7      long id = (long)arg;
8      sem_wait(&rlock);
9      /* the first reader in locks out the writers */
10     if (++readers == 1)
```

```
        sem_wait(&room);
    sem_post(&rlock);

    printf("reader %ld reads %d   (readers inside: %d)\n", id, value,
        readers);
    usleep(200000);

    sem_wait(&rlock);
    /* the last reader out lets the writers in */
    if (--readers == 0)
        sem_post(&room);
    sem_post(&rlock);
    return NULL;
}

void *writer(void *arg) {
    long id = (long)arg;
    sem_wait(&room);      /* alone in the room */
    value += 10;
    printf("writer %ld writes %d  (alone)\n", id, value);
    usleep(200000);
    sem_post(&room);
    return NULL;
}
```

Figure 2.5 shows the output. In this run the three readers got into the room first and read the value 0 together: the number of readers inside goes up to 3. The two writers had to wait until all the readers left, and then they wrote one at a time, each of them alone (10, then 20). The order in which the threads start can differ from one run to another, but a writer is never inside together with a reader or with another writer.

```
reader 1 reads 0   (readers inside: 1)
reader 3 reads 0   (readers inside: 3)
reader 2 reads 0   (readers inside: 2)
writer 1 writes 10  (alone)
writer 2 writes 20  (alone)
```

Figure 2.5: Output of `readers`: the readers are inside together, each writer is alone

2.3.1 Observe Answers (Part 3)

1. **Which two lines of `reader()` make that possible?** The lines `if (++readers == 1)` and `sem_wait(&room);`. Only the first reader takes the semaphore `room`; the next readers find a counter larger than 1, skip `sem_wait(&room)` and go in together. While at least one reader is inside, `room` stays at 0, so a writer blocks in its own `sem_wait(&room)`; and a writer that is inside holds `room`, so the first reader blocks. That is why a writer is always alone. The pair `if (--readers == 0)` and `sem_post(&room);` gives the room back when the last reader leaves.

2. **Why does the counter need its own lock?** Because `++readers` and `--readers` are not one step (load, add, store, like `counter++` in Laboratory Work No. 11), and the test must be done together with the change. Without `rlock` two readers could

change the counter at the same time and lose an update, or both could believe they are the first one. Then the counter is wrong: the second "first" reader blocks forever in `sem_wait(&room)`, or the room is given back while a reader is still inside, so a writer writes during a read, or the room is never given back.

3. **What happens to writer 2 if readers keep arriving?** The counter `readers` never goes back to 0, so the last reader never calls `sem_post(&room)`. Writer 2 waits in `sem_wait(&room)` forever, even though nothing is blocked: this is starvation.

2.4 Part 4: Build a deadlock, then remove it

In the original version of the program (kept as `deadlock_bad.c`) there are two threads and two mutexes, taken in opposite order. Each thread holds what the other one needs.

```
pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void *one(void *arg) {
    pthread_mutex_lock(&A);
    printf("one: I have A, I want B\n");
    sleep(1);
    pthread_mutex_lock(&B);   /* waits for two */
    printf("one: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

void *two(void *arg) {
    pthread_mutex_lock(&B);
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&A);   /* waits for one */
    printf("two: I have both\n");
    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);
    return NULL;
}
```

The program was run with `timeout 5`. Figure 2.6 shows the two lines printed by the threads, then nothing happens for 5 seconds, and the exit code 124 means that `timeout` had to kill the program.

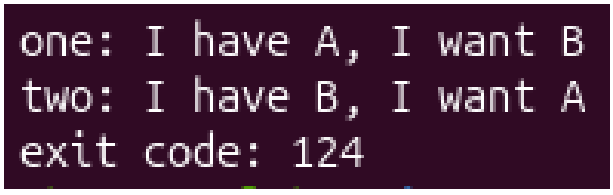


Figure 2.6: The original program is stuck in a deadlock: exit code 124

The fix. In the final version, `deadlock.c`, the function `two()` takes the mutexes in the same order as `one()`: first A, then B. The text printed by the thread was not changed.

```c
void *two(void *arg) {
    pthread_mutex_lock(&A);    /* the fix: lock A first here too */
    printf("two: I have B, I want A\n");
    sleep(1);
    pthread_mutex_lock(&B);    /* then B, the same order as one() */
    printf("two: I have both\n");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}
```

Figure 2.7 shows that the fixed program finishes: one thread takes both mutexes and releases them, then the other one does the same, the program prints `done` and the exit code is 0.

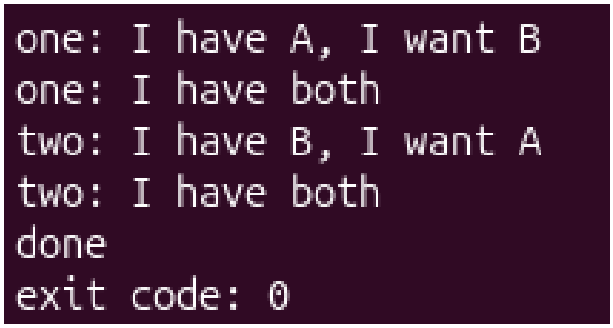


Figure 2.7: The fixed `deadlock` program finishes: exit code 0

For the second question, a copy of the original program without the two `sleep(1)` lines (`deadlock_nosleep.c`) was run ten times, each time with `timeout 2`. Figure 2.8 shows the exit code of every run: 0 means that the run finished and 124 means that it was stuck.

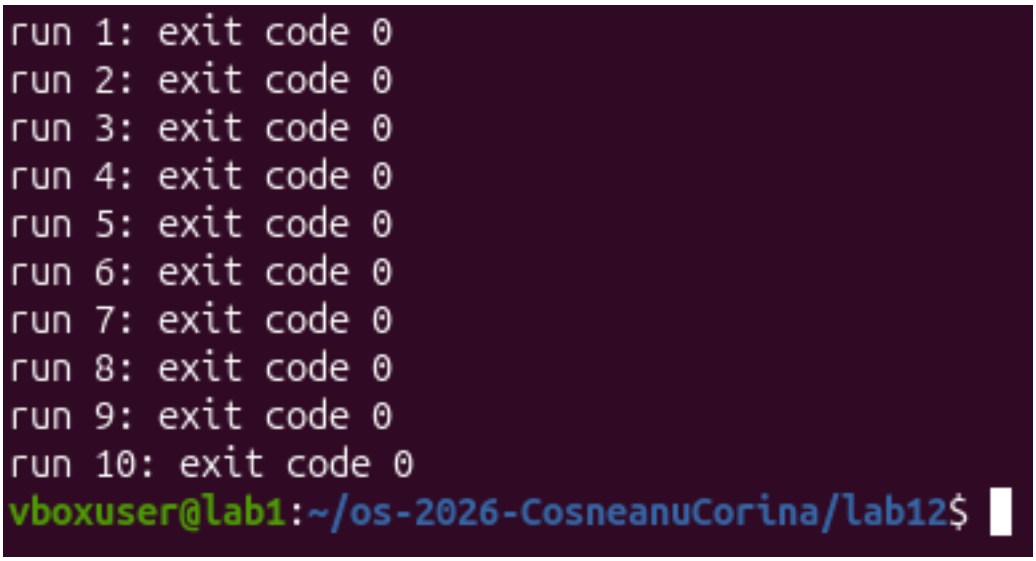


Figure 2.8: Ten runs of the original program without `sleep(1)`: every run finished with the exit code 0

2.4.1 Observe Answers (Part 4)

1. **Who holds what, and who waits for what?** Thread `one` holds mutex A and waits for mutex B. Thread `two` holds mutex B and waits for mutex A. The circle is drawn in Figure 2.9.

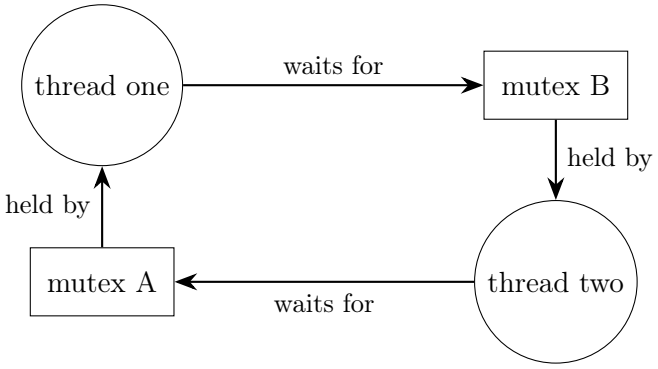


Figure 2.9: The circle of the deadlock: each thread waits for the mutex held by the other

2. **Does it still always hang without `sleep(1)`? Why is that worse?** No. Without the `sleep(1)` lines it did not hang at all in my ten runs: every run finished with the exit code 0 (Figure 2.8). A run blocks only if both threads take their first mutex before either of them takes the second one, and without the sleep the first thread usually takes both mutexes and finishes before the second thread even starts. That is worse, not better: the bug is still in the program, but now it is hidden. It can pass all the tests and then appear very rarely, when the timing is unlucky, and it is very hard to reproduce and to debug.

3. **Why can taking the locks in the same order never deadlock?** If every thread takes the locks in the same order, a thread that waits for a lock can only hold locks that come before it in that order, so a circle of threads waiting for each other can never be formed.

3 CONCLUSIONS

In this laboratory work the synchronization tools were used together. In the producer and consumer problem, two semaphores count the free and the filled slots of the buffer and a mutex protects the buffer itself; the experiment with the swapped lines showed that the order of the calls matters, because a thread that sleeps on a semaphore while holding the mutex blocks the whole program. The condition variable solved the ordering problem of the previous laboratory work in another way: the state is kept in a normal variable, and the condition must be checked again in a `while` loop after every wakeup.

Furthermore, the readers and writers program showed how a counter and two semaphores let many readers work together while a writer stays alone, and why this simple solution can starve the writers. Finally, I built a deadlock with two mutexes taken in opposite order, saw the circle of threads that wait for each other, and removed it by taking the locks in the same order in both threads. The version without `sleep` did not hang in any of the ten runs, although the bug was still in the code. This showed that a deadlock that appears only rarely is the most dangerous kind, because it is hard to notice and to reproduce.